CardioPredict: An AI-Based Heart Disease Risk Prediction System Using Machine Learning

Data Preprocessing and Feature Engineering

Yuva Intern - Week 2



Prepared by:
Venkatachalam K

1. Introduction

CardioPredict is an AI-based heart disease risk prediction system that uses supervised machine learning to estimate whether a patient is likely to have heart disease from routine clinical measurements such as age, blood pressure, cholesterol, and electrocardiogram findings. The system is intended as a decision-support concept and not as a replacement for clinical judgement.

Data preprocessing is the stage of the machine learning workflow in which raw data is inspected, cleaned, and converted into a form that algorithms can use. Model quality depends strongly on input quality: missing entries, duplicate rows, inconsistent encodings, and features on very different numeric scales can all reduce accuracy or make results unreliable.

Medical datasets need particular care. Values are often missing because a test was not performed, extreme measurements may be genuine clinical findings rather than mistakes, and the cost of a misleading model is higher than in many other domains. Every cleaning decision therefore needs to be explained and justified rather than applied mechanically.

This document describes the Week 2 work, in which the raw UCI Cleveland heart disease data is collected, inspected, cleaned, transformed, and prepared as a numeric feature matrix and a binary target vector. These outputs are the inputs for model training in later stages of the project.

2. Objective

The objective of Week 2 is to design and document a data preprocessing and feature engineering pipeline for CardioPredict. The pipeline covers data collection, dataset inspection, missing-value handling, duplicate removal, target transformation, outlier detection and treatment, categorical encoding, numerical scaling, separation of features and target, and data visualization.

The focus of this task is preparing raw data. Developing, tuning, evaluating, or deploying a prediction model is outside the scope of Week 2, and no model performance figures are reported in this document.

3. Dataset Description

The UCI Heart Disease Dataset is a widely used benchmark collection hosted by the UCI Machine Learning Repository. It combines patient records from four medical centres (Cleveland, Hungary, Switzerland, and the VA Long Beach). The original database contains 76 attributes, but published studies and this project use a subset of 14. The Cleveland subset is the most commonly used part because it has the fewest missing values.

Each record describes one patient through demographic details, symptoms, resting measurements, and exercise test results. The target variable records the angiographic diagnosis of heart disease on a scale from 0 (no disease) to 4. The dataset is publicly available and anonymized, so no direct patient contact is needed.

Dataset source: https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data

3.1 Column Descriptions

Column	Meaning	Type	Notes
age	Age of the patient in years	Numerical	Continuous
sex	Sex of the patient (1 = male, 0 = female)	Categorical	Binary
cp	Chest pain type (values 1 to 4)	Categorical	Nominal
trestbps	Resting blood pressure (mm Hg)	Numerical	Continuous
chol	Serum cholesterol (mg/dl)	Numerical	Continuous
fbs	Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)	Categorical	Binary
restecg	Resting electrocardiographic result (0, 1, 2)	Categorical	Nominal
thalach	Maximum heart rate achieved	Numerical	Continuous
exang	Exercise-induced angina (1 = yes, 0 = no)	Categorical	Binary
oldpeak	ST depression induced by exercise relative to rest	Numerical	Continuous
slope	Slope of the peak exercise ST segment (1, 2, 3)	Categorical	Nominal
ca	Number of major vessels (0 to 3) coloured by fluoroscopy	Numerical (count)	Has missing values
thal	Thalassemia result (3 = normal, 6 = fixed defect, 7 = reversible defect)	Categorical	Has missing values
target	Diagnosis of heart disease (original values 0 to 4)	Target	Converted to binary

In this project cp, restecg, slope, and thal are treated as nominal categorical variables and one-hot encoded. The binary columns sex, fbs, and exang are already numeric (0/1) and are used as they are. The continuous measurements age, trestbps, chol, thalach, and oldpeak are the numerical features used for outlier treatment and scaling. The ca column is an integer count that is imputed but is not capped or scaled in this pipeline.

4. Tools and Technologies

Tool	Purpose in this project
Python	General-purpose programming language used to implement the pipeline.
Pandas	Reading data into a DataFrame, inspection, missing-value handling, duplicate removal, and one-hot encoding.
NumPy	Numerical operations, including quantile calculation and clipping of outlier values.
Requests	Downloading the raw data file from the UCI repository over HTTP.
Matplotlib	Creating plots such as histograms and box plots.
Seaborn	Statistical visualization, including count plots and the correlation heatmap.

Tool	Purpose in this project
Scikit-learn	Provides StandardScaler for feature standardization and the preprocessing tools used later in modelling.
Google Colab / Jupyter Notebook	Interactive environment for running code, viewing outputs, and documenting the workflow.

5. Data Collection

The data is downloaded directly from the UCI repository using the Python Requests library. The file processed.cleveland.data is a comma-separated text file with no header row, so the 14 column names are defined manually. In this file, missing values are written as the character '?'; passing na_values='?' to Pandas converts them to NaN so they are recognised as missing and the affected columns keep numeric data types.

```
import io
import requests
import numpy as np
import pandas as pd

url = ("https://archive.ics.uci.edu/ml/machine-learning-databases/"
       "heart-disease/processed.cleveland.data")

# Column names are not included in the file, so they are defined manually
columns = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
           "thalach", "exang", "oldpeak", "slope", "ca", "thal", "target"]

# Download the raw data
response = requests.get(url, timeout=30)
response.raise_for_status()          # stop early if the download failed

# '?' marks missing values in the original file
df = pd.read_csv(io.StringIO(response.text), header=None,
                 names=columns, na_values="?")

print("Dataset shape:", df.shape)
```

The printed shape gives the number of records and columns in the raw dataset and is recorded in Section 15.

6. Dataset Inspection and Exploratory Analysis

Before any change is made, the dataset is inspected to understand its structure and quality. Each check below answers a specific question that guides later cleaning decisions.

- **Shape:** confirms the number of rows and columns and shows how much data is available.
- **Data types:** shows whether columns were read as numeric; unexpected object types usually indicate unhandled placeholders such as '?'.
- **Statistical summary:** shows count, mean, standard deviation, minimum, quartiles, and maximum, which highlights differing scales and suspicious ranges.
- **First records:** a visual check that columns are aligned with their names.

- **Missing values:** counts missing entries per column to decide on a handling strategy.
- **Duplicates:** counts fully repeated rows.
- **Target distribution:** shows the class balance and the original 0 to 4 coding.

```
print(df.shape)                       # rows, columns
print(df.dtypes)                      # data type of each column
print(df.describe())                  # statistical summary
print(df.head())                      # first five records

print(df.isnull().sum())              # missing values per column
print("Duplicates:", df.duplicated().sum())

print(df["target"].value_counts())    # original class distribution (0-4)
```

7. Data Cleaning

7.1 Missing Values

The original file uses '?' to mark values that were not recorded. This is a plain-text convention and not a numeric value, so if it is left unconverted Pandas reads the whole column as text. Converting '?' to NaN, as done during data collection, lets Pandas count and handle missing values correctly.

The ca and thal features contain missing entries in the Cleveland subset. Both are filled with the column median using fillna().

```
# Missing values before imputation
print(df[["ca", "thal"]].isnull().sum())

# Replace missing values with the median of each column
df["ca"] = df["ca"].fillna(df["ca"].median())
df["thal"] = df["thal"].fillna(df["thal"].median())

# Confirm that no missing values remain
print(df.isnull().sum().sum())
```

Median imputation was chosen because it is simple, keeps every record, and is not distorted by extreme values the way the mean can be. It also keeps the values within the range already present in the data.

Its limitations should be understood. The median is a single constant, so it reduces variance and can weaken relationships between features. It ignores other patient information that might predict the missing value. For thal, which is categorical, the median happens to be an existing category code, but the choice implicitly assigns the most central code to patients whose result is unknown. More advanced alternatives such as mode imputation for categorical data, k-nearest-neighbour imputation, or model-based imputation can be considered later.

7.2 Duplicate Records

Duplicate rows are detected with duplicated() and removed with drop_duplicates(). Duplicates can bias a model by giving repeated patients extra weight, and if copies land in both training and test sets they inflate evaluation scores because the model is tested on rows it has already seen.

```
# Count and remove fully duplicated rows
```

```
dup_count = df.duplicated().sum()
print("Duplicate records found:", dup_count)

df = df.drop_duplicates().reset_index(drop=True)
print("Shape after duplicate removal:", df.shape)
```

7.3 Target Variable Transformation

In the original dataset the target holds values from 0 to 4. A value of 0 means no angiographic evidence of heart disease, and values 1 to 4 indicate disease of increasing severity or extent. The project converts this to a binary label:

Original value	New value
0	0 (No disease)
1, 2, 3, 4	1 (Disease present)

Binary classification is used because the project question is whether a patient is at risk or not. The Cleveland subset also has few examples of the higher severity levels, so a five-class problem would leave some classes with very little data. A binary target is simpler to model, evaluate, and interpret.

```
# 0 stays 0 (no disease); 1, 2, 3 and 4 all become 1 (disease present)
df["target"] = (df["target"] > 0).astype(int)

print(df["target"].value_counts())
```

8. Outlier Detection and Treatment

Outlier analysis is applied to the continuous numerical features: age, trestbps, chol, thalach, and oldpeak. Binary and categorical columns are excluded because the concept of an outlier does not apply to coded categories.

Box plots are used first to look at the spread of each feature. A box plot shows the median, the interquartile box, whiskers, and individual points beyond the whiskers, which are candidate outliers.

Candidate outliers are defined with the Interquartile Range (IQR) method. Q1 is the 25th percentile and Q3 is the 75th percentile. The IQR is the spread of the middle 50 percent of the data, and values beyond 1.5 times the IQR from the quartiles fall outside the bounds.

Quantity	Formula
IQR	IQR = Q3 − Q1
Lower Bound	Lower Bound = Q1 − 1.5 × IQR
Upper Bound	Upper Bound = Q3 + 1.5 × IQR

Capping (also called winsorizing) replaces values below the lower bound with the lower bound, and values above the upper bound with the upper bound, using NumPy's clip function. Records are kept and extreme values are limited.

```
num_cols = ["age", "trestbps", "chol", "thalach", "oldpeak"]
```

```
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap values outside the bounds instead of deleting the rows
    df[col] = np.clip(df[col], lower_bound, upper_bound)
```

An important caution applies: an outlier is not automatically an error. In medical data, very high cholesterol, an unusually low maximum heart rate, or a large ST depression may be genuine and clinically meaningful observations, and they may be exactly the cases that relate to disease. Capping reduces the influence of such values on distance-based and gradient-based algorithms, but it also discards some real information. The bounds should therefore be reviewed against clinical knowledge, and it is good practice to compare model results with and without capping.

9. Categorical Encoding

Most machine learning algorithms need numeric input. Nominal categories such as chest pain type are stored as numbers (for example 1 to 4), but these numbers are only labels. If they are used directly, a model may treat code 4 as being twice code 2 or as having a meaningful order, which is not the case for nominal data.

One-hot encoding addresses this by creating a separate 0/1 column for each category. It is applied with the Pandas get_dummies() function to four columns:

- **cp** – chest pain type, four categories.
- **restecg** – resting electrocardiographic result, three categories.
- **slope** – slope of the peak exercise ST segment, three categories.
- **thal** – thalassemia result, three categories (3, 6, 7).

```
cat_cols = ["cp", "restecg", "slope", "thal"]

# One-hot encode; drop_first=True removes one dummy column per feature
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)

print(df_encoded.shape)
print(df_encoded.columns.tolist())
```

The parameter drop_first=True removes the first category of each encoded feature. The dropped category is implied when all remaining dummy columns for that feature are 0. This avoids the dummy variable trap, a perfect redundancy between columns (multicollinearity) that can destabilise linear models such as logistic regression.

As a result, cp (4 categories) becomes 3 columns, and restecg, slope, and thal (3 categories each) become 2 columns each, so 4 original columns are replaced by 9 binary columns. Because thal is stored as a decimal after imputation, the new column names are expected to contain values such as thal_6.0 and thal_7.0. The exact column list should be confirmed from the notebook output.

10. Feature Scaling

Numerical features in this dataset are on very different scales: cholesterol is typically in the hundreds, while oldpeak is a small decimal. StandardScaler rescales each feature so that it has a mean of 0 and a standard deviation of 1, using the formula:

$$z = (x - \mu) / \sigma$$

Here x is the original value, μ (mu) is the mean of the feature, which is the sum of its values divided by their number, and σ (sigma) is the standard deviation, which measures how far values typically lie from the mean. The result z says how many standard deviations a value is above or below the mean.

Scaling helps algorithms that rely on distances or gradients, such as logistic regression, k-nearest neighbours, support vector machines, and neural networks, because features with large numeric ranges would otherwise dominate. Tree-based models such as Random Forest do not require scaling, but scaling does not harm them, and it keeps the processed data usable for several algorithms.

The five continuous features (age, trestbps, chol, thalach, oldpeak) are standardized. The one-hot encoded columns and the binary columns are not, because they are already 0/1 indicators; standardizing them would remove their direct interpretation without benefit.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Standardize only the continuous numerical features
df_encoded[num_cols] = scaler.fit_transform(df_encoded[num_cols])
```

Data leakage note. In the current implementation the scaler is fitted on the complete dataset. This means the mean and standard deviation include information from rows that could later be in a test set, which is a mild form of data leakage. For model training in later weeks, the data should be split into training and test sets first, the scaler (and the median values and IQR bounds) should be learned from the training set only, and the same fitted parameters should then be applied to the test set.

```
# Recommended improvement for the model-training stage (not part of Week 2)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])  # learn from train only
X_test[num_cols] = scaler.transform(X_test[num_cols])        # reuse train parameters
```

11. Feature Engineering and Feature Preparation

This section summarises the implemented transformations and the reasoning behind them. No new derived features (such as ratios or interaction terms) were created in Week 2; the feature engineering consists of transforming existing columns into model-ready form.

11.1 Implemented Transformations

Transformation	Purpose	Expected impact
Binary target creation	Reduces the 0 to 4 diagnosis to disease / no disease.	Simpler problem with more examples per class; easier evaluation.
Categorical encoding	Converts nominal codes to 0/1 columns.	Prevents false ordering; allows algorithms to use these variables.
Numerical scaling	Puts continuous features on a common scale.	Helps distance- and gradient-based models converge and weigh features fairly.
Missing-value handling	Fills gaps in ca and thal with the median.	Retains all records and avoids errors from NaN values.
Outlier capping	Limits extreme values using IQR bounds.	Reduces the influence of extremes; may remove some genuine signal.

11.2 Feature Preparation

After the transformations, the processed table is split into the input feature matrix X and the target vector y so that they can be passed to a model in later stages.

```
# Separate input features (X) from the target label (y)
X = df_encoded.drop("target", axis=1)
y = df_encoded["target"]

print("Feature matrix shape:", X.shape)
print("Target shape:", y.shape)
```

11.3 Possible Future Improvements

The following are suggestions only and were not implemented in Week 2:

- Split the data before fitting any preprocessing step, as described in Section 10.
- Use a scikit-learn Pipeline and ColumnTransformer so that all steps are applied consistently.
- Try more advanced imputation methods and compare model results with and without outlier capping.
- Evaluate derived clinical features and feature selection once a baseline model exists.
- Use cross-validation to obtain more reliable estimates on a small dataset.

12. Data Visualization

Visualizations are used to understand the data and to support preprocessing choices. The code below uses the cleaned DataFrame df. Descriptions state what each chart is designed to show; actual findings should be written only after viewing the generated output.

The notebook-level imports for plotting are:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

12.1 Heart Disease Target Distribution

Purpose: shows how many records fall into each class after the binary transformation. It indicates whether the classes are balanced, which affects the choice of evaluation metrics and whether stratified splitting or class weighting may be needed.

```
plt.figure(figsize=(6, 4))
sns.countplot(x="target", data=df)
plt.title("Heart Disease Target Distribution")
plt.xlabel("Target (0 = No disease, 1 = Disease present)")
plt.ylabel("Number of patients")
plt.show()
```

[Insert Target Distribution Plot Here]

12.2 Age Distribution Histogram

Purpose: shows how patient ages are spread across the dataset, including the general shape, range, and any concentration of patients in particular age bands. It helps judge whether the sample is representative and whether age is roughly symmetric before scaling.

```
plt.figure(figsize=(7, 4))
plt.hist(df["age"], bins=15, edgecolor="black")
plt.title("Age Distribution of Patients")
plt.xlabel("Age (years)")
plt.ylabel("Frequency")
plt.show()
```

[Insert Age Distribution Histogram Here]

12.3 Box Plots of Numerical Features

Purpose: shows the median, spread, and points beyond the whiskers for each continuous feature. It identifies candidate outliers and guides the decision to apply IQR-based capping. Because the features are on different scales, each is plotted in its own panel. Plots produced before and after capping can be compared to see the effect of the treatment.

```
fig, axes = plt.subplots(1, len(num_cols), figsize=(15, 4))
for ax, col in zip(axes, num_cols):
    sns.boxplot(y=df[col], ax=ax)
    ax.set_title(col)
plt.tight_layout()
plt.show()
```

[Insert Numerical Features Box Plot Here]

12.4 Correlation Heatmap

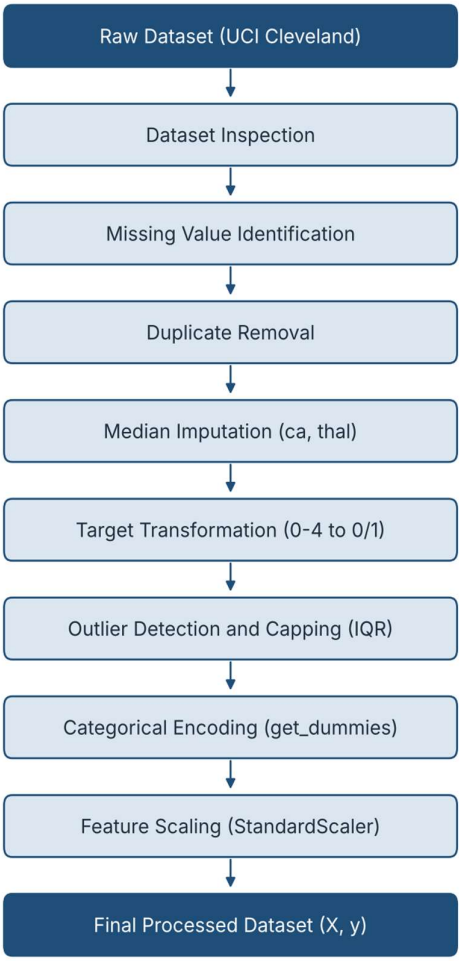
Purpose: displays pairwise Pearson correlation coefficients between numeric columns as a colour-coded grid. It shows which features move together and which are most associated with the target, and it can reveal strongly correlated feature pairs that may be redundant. Correlation measures linear association only and does not prove cause and effect.

```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

[Insert Correlation Heatmap Here]

13. Complete Preprocessing Pipeline

The complete sequence of Week 2 operations is shown below. Each step takes the output of the previous step as its input.



Step	Operation	Output
1	Raw dataset	Downloaded data with '?' converted to NaN
2	Dataset inspection	Shape, types, summary, missing and duplicate counts
3	Missing value identification	Counts of missing entries per column
4	Duplicate removal	Dataset without repeated rows
5	Median imputation	ca and thal without missing values

Step	Operation	Output
6	Target transformation	Binary target (0/1)
7	Outlier detection and capping	Numerical features limited to IQR bounds
8	Categorical encoding	One-hot columns for cp, restecg, slope, thal
9	Feature scaling	Standardized continuous features
10	Final processed dataset	Feature matrix X and target y

14. Coding

```
import pandas as pd
import requests
import io
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Configure visualization style
sns.set_theme(style="whitegrid")

# 1. The official URL for the clean UCI Cleveland Heart Disease dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data"

# 2. Fetch the data using requests (disabling strict SSL verification)
response = requests.get(url, verify=False)

# 3. Define the standard 14 column names
column_names = [
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
    'restecg', 'thalach', 'exang', 'oldpeak', 'slope',
    'ca', 'thal', 'target'
]

# 4. Load the text content into Pandas using io.StringIO
df = pd.read_csv(io.StringIO(response.text), header=None, names=column_names, na_values='?')

print(f"Dataset Loaded Successfully! Shape: {df.shape}")
```

```
# --- STEP 2: DATA CLEANING & MISSING VALUES ---
print("\n--- Missing Values Per Column ---")
print(df.isnull().sum())

print(f"\nDuplicate rows found: {df.duplicated().sum()}")
df.drop_duplicates(inplace=True)

# Impute missing values using median
for col in ['ca', 'thal']:
    if col in df.columns and df[col].isnull().sum() > 0:
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"Imputed missing values in '{col}' using median: {median_val}")

# Convert target into binary classification (0 = Healthy, 1 = At Risk)
df['target'] = df['target'].apply(lambda x: 1 if x > 0 else 0)

print("\nUpdated Target Class Distribution:")
print(df['target'].value_counts())

# --- STEP 3: OUTLIER DETECTION & TREATMENT ---
print("\n--- Running Outlier Treatment ---")
numerical_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

# Optional: Uncomment the next 6 lines if you want the box plots to pop up visually
# plt.figure(figsize=(12, 6))
# for i, col in enumerate(numerical_cols, 1):
#     plt.subplot(2, 3, i)
#     sns.boxplot(y=df[col], color='skyblue')
#     plt.title(f'Box Plot of {col}')
# plt.tight_layout()
# plt.show()

# Function to cap outliers using IQR
def cap_outliers(data, column):
    Q1 = data[column].quantile(0.25)
```

```
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    data[column] = np.where(data[column] < lower_bound, lower_bound, data[column])
    data[column] = np.where(data[column] > upper_bound, upper_bound, data[column])
    print(f"Capped outliers for '{column}'")

for col in ['trestbps', 'chol', 'thalach', 'oldpeak']:
    cap_outliers(df, col)

print("\nStep 3 Outlier Capping Complete!")

# --- STEP 4: CATEGORICAL ENCODING & FEATURE SCALING ---
from sklearn.preprocessing import StandardScaler

print("\n--- Running Encoding & Feature Scaling ---")

# 1. One-hot encode categorical variables with multiple categories
categorical_cols = ['cp', 'restecg', 'slope', 'thal']
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# 2. Separate features (X) and target (y)
X = df.drop(columns=['target'])
y = df['target']

# 3. Scale continuous numerical columns to normalize variance
scaler = StandardScaler()
continuous_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
X[continuous_cols] = scaler.fit_transform(X[continuous_cols])

print(f"Final Processed Feature Matrix Shape: {X.shape}")
print("\nPipeline execution complete! Ready for model training.")

# --- GENERATE & SAVE OUTLIER BOX PLOTS ---
print("\nGenerating and saving outlier box plots...")
```

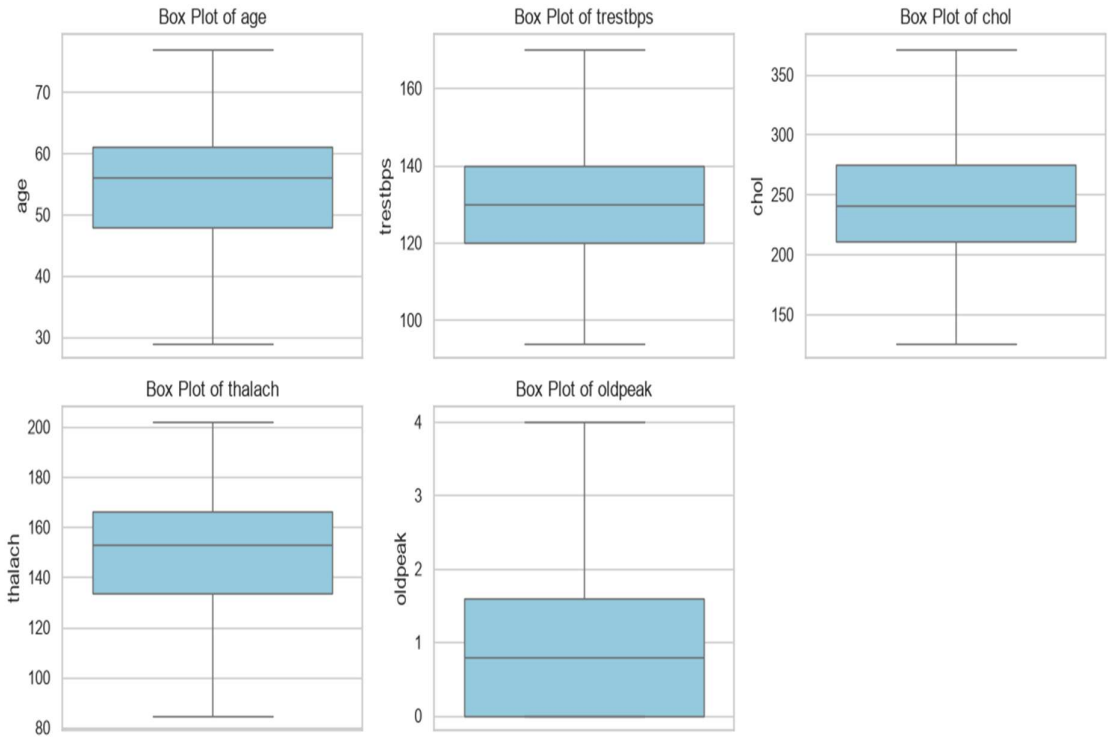
```
numerical_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

plt.figure(figsize=(12, 6))
for i, col in enumerate(numerical_cols, 1):
    plt.subplot(2, 3, i)
    sns.boxplot(y=df[col], color='skyblue')
    plt.title(f'Box Plot of {col}')

plt.tight_layout()

# Save the plot as an image file in your project folder
plt.savefig('outlier_boxplots.png')
plt.close() # Close the plot to free up memory
print("Visualization saved successfully as 'outlier_boxplots.png' in your project folder!")
```

15. Visual Box Plot



16. Challenges and Limitations

- **Missing values in medical data:** missing entries often reflect clinical decisions about which tests were performed, so they may not be random. Simple imputation may hide this pattern.
- **Small dataset size:** the Cleveland subset has only a few hundred records, which limits the complexity of models that can be trained reliably and makes results sensitive to how the data is split.
- **Risk of overfitting:** with few records and many encoded columns, a flexible model can memorise training data. Cross-validation and regularisation will be needed.
- **Information loss from outlier capping:** genuine extreme clinical values are flattened to the bound, which may remove useful signal.
- **Limits of median imputation:** it reduces variability and ignores relationships with other features.
- **Data leakage:** fitting the scaler (and computing medians and IQR bounds) on the full dataset lets information from future test records influence preprocessing. This should be corrected by splitting first.
- **Benchmark versus real-world populations:** the data comes from a limited group of patients collected decades ago at specific centres. Models trained on it may not generalise to other populations, devices, or clinical settings and are not suitable for clinical use without proper validation.

18. Results and Output

Running the preprocessing pipeline is expected to produce a dataset with no missing values, no duplicate rows, a binary target, extreme numerical values limited to IQR bounds, nominal variables expressed as 0/1 columns, and continuous features with approximately zero mean and unit standard deviation. The table below lists the values that must be filled in from the actual notebook output. No figures have been assumed here.

Item	Value	Source
Original dataset shape	[Fill from notebook]	Section 5
Missing values before cleaning	[Fill from notebook]	Section 7.1
Missing values after cleaning	[Fill from notebook]	Section 7.1
Duplicate records removed	[Fill from notebook]	Section 7.2
Final processed feature matrix shape (X)	[Fill from notebook]	Section 11.2
Final target distribution (0 / 1)	[Fill from notebook]	Section 7.3

18. Conclusion

The Week 2 pipeline converts the raw UCI Cleveland heart disease data into a clean, fully numeric dataset that is ready for machine learning. Missing values were handled, duplicate records removed, the diagnosis was converted to a binary target, extreme values were limited with the IQR method, nominal variables were one-hot encoded, and continuous features were standardized. The resulting feature matrix X and target y give a consistent starting point for model development.

The document also notes the main weaknesses of the current approach, in particular fitting preprocessing on the complete dataset, which should be corrected by splitting the data before model training. With that improvement, the pipeline provides a sound basis for the training and evaluation work in the following weeks.

19. References

1. UCI Machine Learning Repository – Heart Disease Dataset. https://archive.ics.uci.edu/dataset/45/heart+disease
2. Pandas documentation. https://pandas.pydata.org/docs/
3. NumPy documentation. https://numpy.org/doc/
4. Scikit-learn: StandardScaler. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
5. Scikit-learn: Preprocessing data. https://scikit-learn.org/stable/modules/preprocessing.html
6. Matplotlib documentation. https://matplotlib.org/stable/
7. Seaborn documentation. https://seaborn.pydata.org/